Chapter 26 | Exercise solutions with Python

Models for Code

This notebook solves Exercises 5–8. It follows individual test outcomes from the chapter's interval benchmark, adds a case that exposes a boundary assumption, and contrasts candidate coverage with a fixed selection rule.

The examples run locally on checked-in Python functions. Each section derives the expected result before comparing it with the observed outcomes. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python 3 kernel. The exercises use the Python standard library. Run the cells in order to reproduce the saved outputs.

```
from pathlib import Path
import sys

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch26_code_models"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Inspect the evaluation record

Run the chapter's listing in the companion repository. The `--output` argument saves the evaluation as JSON. A temporary directory gives us a fresh file to read on each run.

Under `benchmark.candidates`, each candidate has one record per edge case in `edges.per_case`. Read `expected` for the answer set by the contract, `observed` for the candidate's answer, and `passed` for whether the scorer accepted it. Inspect `touching` for `touching_counts` and `length_one` for `needs_two`.

```
import json
import subprocess
from tempfile import TemporaryDirectory

with TemporaryDirectory() as temporary_directory:
    output_path = Path(temporary_directory) / "ch26.json"
    subprocess.run(
        [
            sys.executable,
            "-m", "examples.ch26_code_models.run_examples",
            "--output", str(output_path),
        ],
        cwd=repo_root,
        check=True,
        capture_output=True,
    )
    evaluation = json.loads(output_path.read_text(encoding="utf-8"))

candidate_records = {
    record["name"]: record
    for record in evaluation["benchmark"]["candidates"]
}
selected_cases = (
    ("touching_counts", "touching", False, True),
    ("needs_two", "length_one", True, False),
)
for name, case_name, expected, observed in selected_cases:
    case_records = candidate_records[name]["edges"]["per_case"]
    record = next(row for row in case_records if row["case"] == case_name)
    assert record["expected"] is expected
    assert record["observed"] is observed
    assert record["passed"] is False
    print(f"{name} on {case_name}")
    print(
        f"  expected {record['expected']}, "
        f"observed {record['observed']}, "
        f"passed {record['passed']}"
    )
```

```
OUTPUT
touching_counts on touching
  expected False, observed True, passed False
needs_two on length_one
  expected True, observed False, passed False
```

For `touching`, the intervals are $[0, 2)$ and $[2, 5)$. The first stops before 2, so the two intervals share no integers and the expected result is `False`. The later start is $ell = 2$ and the earlier end is $r = 2$. The comparison $ellle r$ evaluates to `True` because it accepts equality.

For `length_one`, the intervals are $[0, 3)$ and $[2, 4)$. Both contain 2, so the expected result is `True`. The later start is $ell = 2$ and the earlier end is $r = 3$. The comparison $r - ellge 2$ evaluates to `False` because the intersection has just one integer.

6. Add one boundary case

Choose $[-5, -2)$ and $[-3, 0)$. The first interval contains $-5, -4, -3$, and the second contains $-3, -2, -1$. The only shared integer is -3, so the expected result is `True`.

Both starts are negative. The later start is $ell = -3$ and the earlier end is $r = -2$, so $r - ell = 1$. Check the intersection independently by listing each interval's integers with `range`, then give the same one-element case list to all four functions from the chapter.

```
from examples.ch26_code_models.model import Case, score_candidate
from examples.ch26_code_models.run_examples import CANDIDATES

negative_start_case = Case(
    "negative_start_length_one", (-5, -2), (-3, 0), True
)
shared = (
    set(range(*negative_start_case.a))
    & set(range(*negative_start_case.b))
)
assert shared == {-3}
assert negative_start_case.expected is bool(shared)
print(f"Shared integers {sorted(shared)}")
print(f"Expected result {negative_start_case.expected}")

outcomes = {}
for candidate in CANDIDATES:
    score = score_candidate(candidate, [negative_start_case])
    assert score["total_cases"] == 1
    record = score["per_case"][0]
    name = candidate.__name__
    outcomes[name] = (record["observed"], record["passed"])
    print(
        f"{name:<16} observed {str(record['observed']):<5} "
        f"agrees with contract {record['passed']}"
    )

assert outcomes == {
    "touching_counts": (True, True),
    "strict_overlap": (True, True),
    "needs_two": (False, False),
    "positive_length": (True, True),
}
```

```
OUTPUT
Shared integers [-3]
Expected result True
touching_counts  observed True  agrees with contract True
strict_overlap   observed True  agrees with contract True
needs_two        observed False agrees with contract False
positive_length  observed True  agrees with contract True
```

`strict_overlap` and `positive_length` return `True` because $-3 < -2$ and $1 > 0$. `touching_counts` also returns `True` here, since $-3le-2$. The touching case in Exercise 5 exposes its error when $ell = r$. Only `needs_two` disagrees with the contract on the new case, since $1ge2$ is false.

7. Check the metric

Label the five candidates A through E. A and B pass the tests. We choose k different candidates from this pool, with every size-k subset equally likely. The subset succeeds if it contains A or B. Here $n = 5$ is the number of candidates and $c = 2$ is the number that pass.

For $k = 2$, Exercise 4 counted ten pairs. The pairs CD, CE, and DE contain only failures. That leaves seven successful pairs, so `pass_at_k(5, 2, 2)` should return 7/10.

The chapter function counts subsets using the formula from the text. We check it independently with `combinations`, which lists each subset once. The fraction keeps the enumeration exact.

```
from fractions import Fraction
from itertools import combinations

from examples.ch26_code_models.model import pass_at_k

candidate_names = "ABCDE"
passing_names = {"A", "B"}

for k in (1, 2, 5):
    subsets = list(combinations(candidate_names, k))
    successes = sum(
        any(name in passing_names for name in subset)
        for subset in subsets
    )
    exact_coverage = Fraction(successes, len(subsets))
    coverage = pass_at_k(len(candidate_names), len(passing_names), k)
    assert coverage == float(exact_coverage)
    print(
        f"k={k}: {successes}/{len(subsets)} subsets succeed, "
        f"pass@{k} = {coverage:.2f}"
    )
```

```
OUTPUT
k=1: 2/5 subsets succeed, pass@1 = 0.40
k=2: 7/10 subsets succeed, pass@2 = 0.70
k=5: 1/1 subsets succeed, pass@5 = 1.00
```

With $k = 1$, two of the five one-candidate subsets pass, so $pass@1 = 2/5$. With $k = 2$, at least one of two distinct candidates passes in seven of the ten equally likely pairs, so $pass@2 = 7/10$. With $k = 5$, there is only one subset. It contains all five candidates, including both passes, so $pass@5 = 1$.

8. Separate selection from coverage

Call the 12-line, 18-line, and 7-line candidates A, B, and C. A and B pass the tests. The selector compares line counts and returns C because it has just seven lines. To score the rule, check the result of the candidate it returned. C fails, giving the rule a score of 0 on this task.

For *pass*@2, look at the three possible pairs. AB contains two passes, AC contains one, and BC contains one. Each pair succeeds because it contains at least one passing candidate. The pool therefore has *pass*@2 = 3/3 = 1.

```
from examples.ch26_code_models.model import pass_at_k

# Each record contains the candidate's name, line count, and test result.
candidates = (
    ("A", 12, True),
    ("B", 18, True),
    ("C", 7, False),
)

selected_name, line_count, passed = min(
    candidates, key=lambda candidate: candidate[1]
)
selection_score = int(passed)
passing_count = sum(passed for _, _, passed in candidates)
pool_coverage = pass_at_k(len(candidates), passing_count, 2)

assert selected_name == "C" and selection_score == 0
assert pool_coverage == 1.0

print(f"Selected candidate: {selected_name} ({line_count} lines)")
print(f"Shortest rule score: {selection_score}")
print(f"Uniform pool pass@2: {pool_coverage:.2f}")
```

```
OUTPUT
Selected candidate: C (7 lines)
Shortest rule score: 0
Uniform pool pass@2: 1.00
```

The user receives C and sees a failing answer. Across many tasks, the quantity that describes what users receive is the fraction of tasks on which the candidate selected by the rule passes. The pool's *pass*@2 gives the chance that two uniformly chosen candidates include a passing answer.